Lecture 05.16: Polymorphism — Duck Typing & Dunder Methods

📌 Executive Summary

Duck typing philosophy ('If it walks like a duck...'), special dunder methods (**str**, **repr**, **add**, **len**), and operator overloading.

🔬 Theoretical Foundations & Principles

1. The Duck Typing Axiom

In Python, an object's suitability is determined by the presence of specific methods and properties, rather than by explicit inheritance:

> *"If it walks like a duck and quacks like a duck, it's a duck."*

2. Dunder Methods & Operator Overloading

Python intercepts syntactic operators via special double-underscore methods:

- `+` $\implies `__add__(self, other)`
- `==` $\implies `__eq__(self, other)`
- `len(obj)` $\implies `__len__(self)`
- `str(obj)` $\implies `__str__(self)`
- `repr(obj)` $\implies `__repr__(self)`

💻 Production Implementation & Code

```
class ComplexNumber:
    def __init__(self, real: float, imag: float):
        self.real = real
        self.imag = imag

    def __add__(self, other: "ComplexNumber") -> "ComplexNumber":
        return ComplexNumber(self.real + other.real, self.imag + other.imag)
```

```
    def __repr__(self) -> str:
        return f"ComplexNumber({self.real}, {self.imag})"

    def __str__(self) -> str:
        return f"{self.real} + {self.imag}j"

c1 = ComplexNumber(3, 4)
c2 = ComplexNumber(1, 2)
c3 = c1 + c2  # Operator overloading via __add__
print("c1 + c2 =", c3)
```

⚙️ Algorithmic Complexity & Systems Analysis

- **Time Complexity:** Method dispatch overhead is $\mathcal{O}(1)$ via CPython descriptor resolution.
- **Auxiliary Space:** Instance attribute memory allocation in `__dict__` vs `__slots__`.